Docker 容器部署方法说明

Revision History

Draft Date	Revision No.	Description
2024/10/22	V1.0	1. 初始版本。

目　　录

前　言 3
1 Docker 架构 4
2 Docker 安装 4
2.1　安装方法 4
2.2　搭建本地镜像仓库 7
3 构建镜像 9
3.1　构建单个镜像 9
3.2　构建多个镜像 13
4 下载镜像并运行容器 16
4.1　下载单个镜像并运行容器 16
4.2　下载多个镜像并运行容器 22
5 查看磁盘占用资源情况 25
5.1　未下载 Docker 镜像 25
5.2　已下载 Docker 镜像 26
更多帮助 27

前 言

本指导文档适用开发环境：

Windows 开发环境：Windows 7 64bit、Windows 10 64bit

Linux 开发环境：VMware15.5.5、Ubuntu20.04.6 64bit

U-Boot：U-Boot-2022.04

Kernel：Linux-5.15.71-rt51

LinuxSDK：Real-Time Edge Software 2.5

Docker 是一个开源的应用容器引擎，让开发者可打包他们的应用以及依赖包至一个可移植的镜像中，然后发布至任何流行的 Linux 或 Windows 机器上，亦可实现虚拟化。容器是完全使用沙箱机制，相互之间不会有任何接口。使用 Docker，可像管理应用程序一样管理基础结构。通过利用 Docker 的快速发布、测试和部署代码的方法，可显著减少产品开发时间。

1 Docker 架构

Docker 使用客户端——服务器(C/S)体系结构。Docker 客户端与 Docker 守护进程进行通信，后者执行构建、运行和分发 Docker 容器的繁重工作。Docker 客户端和守护进程可在同一个系统上运行，亦可将 Docker 客户端连接至远程 Docker 守护进程。Docker 客户端和守护进程使用 REST API、UNIX 套接字或网络接口进行通信。

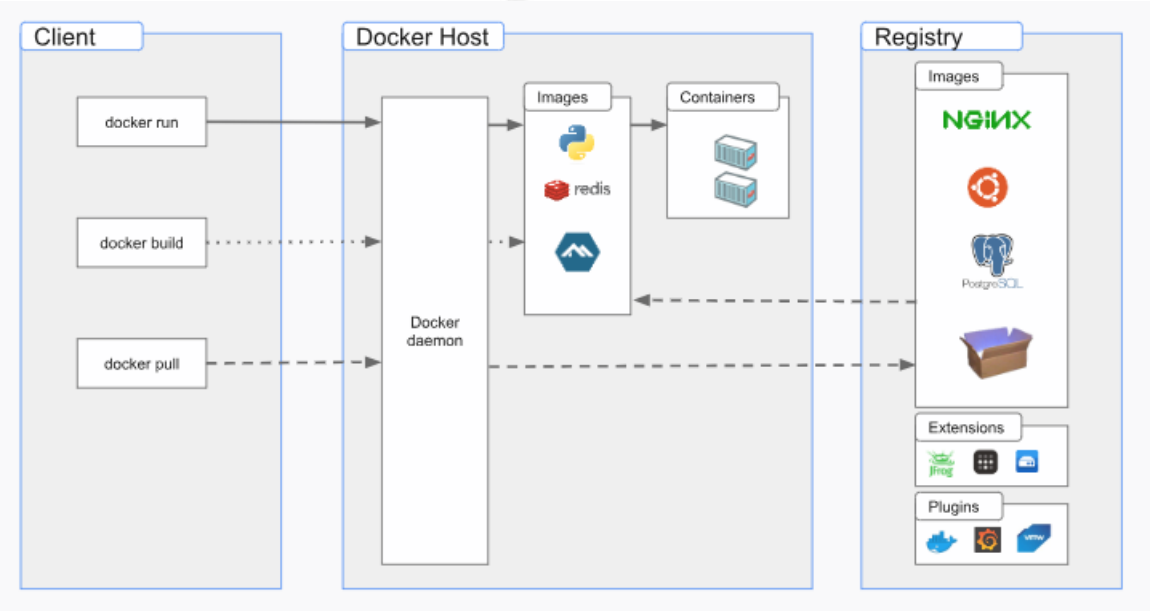

图 1

更多详情请可参考 Docker 官网链接：https://docs.docker.com/get-started/overview。

2 Docker 安装

2.1 安装方法

（1） 安装依赖包

打开 Ubuntu，确保 Ubuntu 可正常访问互联网，执行如下命令进行安装 Docker 依赖包。

Host# sudo apt-get update

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get update
[sudo] password for tronlong:
Hit:1 http://mirrors.aliyun.com/ubuntu focal InRelease
Hit:2 http://mirrors.aliyun.com/ubuntu focal-updates InRelease
Hit:3 http://mirrors.aliyun.com/ubuntu focal-backports InRelease
Hit:4 http://mirrors.aliyun.com/ubuntu focal-security InRelease
Hit:5 http://ppa.launchpad.net/git-core/ppa/ubuntu focal InRelease
Reading package lists... Done
tronlong@tronlong-virtual-machine:~$
```

图 2

Host# sudo apt-get install ca-certificates curl gnupg lsb-release

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get install ca-certificates curl gnupg lsb-release
Reading package lists... Done
Building dependency tree
Reading state information... Done
lsb-release is already the newest version (11.1.0ubuntu2).
lsb-release set to manually installed.
ca-certificates is already the newest version (20230311ubuntu0.20.04.1).
ca-certificates set to manually installed.
gnupg is already the newest version (2.2.19-3ubuntu2.2).
gnupg set to manually installed.
The following NEW packages will be installed:
  curl
0 upgraded, 1 newly installed, 0 to remove and 60 not upgraded.
Need to get 161 kB of archives.
After this operation, 413 kB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://mirrors.aliyun.com/ubuntu focal-updates/main amd64 curl amd64 7.68.0-1ubuntu2.20 [1
61 kB]
Fetched 161 kB in 0s (552 kB/s)
Selecting previously unselected package curl.
(Reading database ... 207898 files and directories currently installed.)
Preparing to unpack .../curl_7.68.0-1ubuntu2.20_amd64.deb ...
Unpacking curl (7.68.0-1ubuntu2.20) ...
Setting up curl (7.68.0-1ubuntu2.20) ...
Processing triggers for man-db (2.9.1-1) ...
tronlong@tronlong-virtual-machine:~$
```

图 3

（2） 添加 Docker 官方 GPG 密钥

Host# sudo mkdir -m 0755 -p /etc/apt/keyrings

Host# curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

```
tronlong@tronlong-virtual-machine:~$ sudo mkdir -m 0755 -p /etc/apt/keyrings
tronlong@tronlong-virtual-machine:~$ curl -fsSL https://mirrors.aliyun.com/docker-ce/linux/ubuntu/gpg | sud
o gpg --dearmor -o /etc/apt/keyrings/docker.gpg
tronlong@tronlong-virtual-machine:~$
```

图 4

（3） 设置 Docker Stable 存储库

Host# echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://mirrors.aliyun.com/docker-ce/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

```
tronlong@tronlong-virtual-machine:~$ echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrin
gs/docker.gpg] https://mirrors.aliyun.com/docker-ce/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc
/apt/sources.list.d/docker.list > /dev/null
tronlong@tronlong-virtual-machine:~$
```

图 5

（4） 安装 Docker

Host# sudo apt-get update

Host# sudo apt-get install docker-ce docker-ce-cli containerd.io

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get update
Hit:1 http://mirrors.aliyun.com/ubuntu focal InRelease
Hit:2 http://mirrors.aliyun.com/ubuntu focal-updates InRelease
Hit:3 http://mirrors.aliyun.com/ubuntu focal-backports InRelease
Hit:4 http://mirrors.aliyun.com/ubuntu focal-security InRelease
Hit:5 http://ppa.launchpad.net/git-core/ppa/ubuntu focal InRelease
Hit:6 https://download.docker.com/linux/ubuntu focal InRelease
Reading package lists... Done
tronlong@tronlong-virtual-machine:~$ sudo apt-get install docker-ce docker-ce-cli containerd.io
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  docker-buildx-plugin docker-ce-rootless-extras docker-compose-plugin pigz slirp4netns
Suggested packages:
  aufs-tools cgroupfs-mount | cgroup-lite
The following NEW packages will be installed:
  containerd.io docker-buildx-plugin docker-ce docker-ce-cli docker-ce-rootless-extras
  docker-compose-plugin pigz slirp4netns
0 upgraded, 8 newly installed, 0 to remove and 60 not upgraded.
Need to get 114 MB of archives.
After this operation, 409 MB of additional disk space will be used.
Do you want to continue? [Y/n] Y
Get:1 http://mirrors.aliyun.com/ubuntu focal/universe amd64 pigz amd64 2.4-1 [57.4 kB]
Get:2 http://mirrors.aliyun.com/ubuntu focal/universe amd64 slirp4netns amd64 0.4.3-1 [74.3 kB]
Get:3 https://download.docker.com/linux/ubuntu focal/stable amd64 containerd.io amd64 1.6.24-1 [
28.6 MB]
Get:4 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-buildx-plugin amd64 0.1
1.2-1~ubuntu.20.04~focal [28.2 MB]
Get:5 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-ce-cli amd64 5:24.0.7-1
~ubuntu.20.04~focal [13.3 MB]
Get:6 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-ce amd64 5:24.0.7-1~ubu
ntu.20.04~focal [22.6 MB]
Get:7 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-ce-rootless-extras amd6
4 5:24.0.7-1~ubuntu.20.04~focal [9,037 kB]
Get:8 https://download.docker.com/linux/ubuntu focal/stable amd64 docker-compose-plugin amd64 2.
```

图 6

```
Setting up slirp4netns (0.4.3-1) ...
Setting up docker-buildx-plugin (0.11.2-1~ubuntu.20.04~focal) ...
Setting up containerd.io (1.6.24-1) ...
Created symlink /etc/systemd/system/multi-user.target.wants/containerd.service → /lib/systemd/sy
stem/containerd.service.
Setting up docker-compose-plugin (2.21.0-1~ubuntu.20.04~focal) ...
Setting up docker-ce-cli (5:24.0.7-1~ubuntu.20.04~focal) ...
Setting up pigz (2.4-1) ...
Setting up docker-ce-rootless-extras (5:24.0.7-1~ubuntu.20.04~focal) ...
Setting up docker-ce (5:24.0.7-1~ubuntu.20.04~focal) ...
Created symlink /etc/systemd/system/multi-user.target.wants/docker.service → /lib/systemd/system
/docker.service.
Created symlink /etc/systemd/system/sockets.target.wants/docker.socket → /lib/systemd/system/doc
ker.socket.
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for systemd (245.4-4ubuntu3.20) ...
tronlong@tronlong-virtual-machine:~$
```

图 7

2.2 搭建本地镜像仓库

Docker 用 Registry 来保存用户构建的镜像。Registry 分为公共和私有两种。Docker 公司运营的公共 Registry 叫做 Docker Hub，用户可在 Docker Hub 注册账号，分享并保存自己的镜像。Docker 公司的公共镜像仓库(https://hub.docker.com)提供了庞大的镜像集合供用户使用。一个 Docker Registry 中可包含多个仓库，每个仓库可包含多个标签(Tag)，每个标签对应一个镜像。通常，一个仓库会包含同一个软件不同版本的镜像，而标签对应该软件的各个版本。用户可通过“[<仓库名>:<标签>]”的格式来指定具体是某个软件某个版本的镜像。如未给出标签，将以[latest]作为默认标签。

执行如下命令创建 myregistry 目录。

Host# mkdir -p /home/tronlong/docker/myregistry

```
tronlong@tronlong-virtual-machine:~$ mkdir -p /home/tronlong/docker/myregistry
tronlong@tronlong-virtual-machine:~$
```

图 8

将产品资料“4-软件资料\Demo\platform-demos\docker\image\”目录下的 Docker 镜像压缩包文件 registry_image.tar 拷贝至 Ubuntu 的"/home/tronlong/docker/myregistry/"目录下，并执行如下命令将其加载至本地仓库。

Host# sudo docker load -i /home/tronlong/docker/myregistry/registry_image.tar

```
tronlong@tronlong-virtual-machine:~/docker/myregistry$ sudo docker load -i /home/tronlong/docker/myregistry
/registry_image.tar
aedc3bda2944: Loading layer   7.63MB/7.63MB
3f2494422d07: Loading layer  771.6kB/771.6kB
09a31214b461: Loading layer  17.55MB/17.55MB
2bcfab93d849: Loading layer  4.096kB/4.096kB
3bb6c126c74a: Loading layer  2.048kB/2.048kB
Loaded image: registry:2
tronlong@tronlong-virtual-machine:~/docker/myregistry$
```

图 9

执行如下命令查看加载的 Registry 镜像。

Host# sudo docker images

```
tronlong@tronlong-virtual-machine:~/docker/myregistry$ sudo docker images
REPOSITORY    TAG       IMAGE ID       CREATED        SIZE
registry      2         9363667f8aec   6 months ago   25.4MB
tronlong@tronlong-virtual-machine:~/docker/myregistry$
```

图 10

本次使用的镜像是 registry:2，请执行如下命令启动容器。

Host# sudo docker run -d -p 5000:5000 -v /home/tronlong/docker/myregistry:/var/lib/registry registry:2

参数说明：

-d：后台启动容器。

-p：将容器的 5000 端口映射至 Ubuntu 的 5000 端口（5000 是 registry 服务端口）。

-v：将容器"/var/lib/registry"目录映射至 Ubuntu 的"/home/tronlong/docker/myregistry"，用于存放镜像数据。

```
tronlong@tronlong-virtual-machine:~/docker/myregistry$ sudo docker run -d -p 5000:5000 -v /home/tronlong/do
cker/myregistry:/var/lib/registry registry:2
e2bc50c046d4437555f8ddf29f2f133e2994415832e573bd04b4cf33ca85db3c
tronlong@tronlong-virtual-machine:~/docker/myregistry$
```

图 11

请执行如下命令，查看容器是否已启动，出现如下内容说明容器已正常启动。

Host# sudo docker ps

```
tronlong@tronlong-virtual-machine:~/docker/myregistry$ sudo docker ps
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS          PORTS                                       NAMES
e2bc50c046d4   registry:2   "/entrypoint.sh /etc…"   28 seconds ago   Up 27 seconds   0.0.0.0:5000->5000/tcp, :::5000->5000/tcp   gracious_leakey
tronlong@tronlong-virtual-machine:~/docker/myregistry$
```

图 12

请通过浏览器访问"http://192.168.13.105:5000/v2/_catalog"，如返回如下内容则说明 registry 已经成功启动，192.168.13.105 为 Ubuntu 的 IP。

图 13

因为 Docker 刚建立运行，所以里面无任何镜像内容。至此，Docker 环境安装完成。

3 构建镜像

3.1 构建单个镜像

请在 Ubuntu 上新建工作目录"/home/tronlong/docker/dockerfile/"，并将产品资料“4-软件资料\Linux\Filesystem\image\”目录下的文件系统压缩包和产品资料“4-软件资料\Demo\base-demos\led_flash\bin\”目录下的可执行文件拷贝至 Ubuntu 共享目录下，再执行如下命令拷贝至 Docker 工作目录。

Host# mkdir /home/tronlong/docker/dockerfile

Host# cp /mnt/hgfs/SharedFolders/led_flash /home/tronlong/docker/dockerfile/

Host# cp /mnt/hgfs/SharedFolders/nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz /home/tronlong/docker/dockerfile/

```
tronlong@tronlong-virtual-machine:~$ mkdir /home/tronlong/docker/dockerfile
tronlong@tronlong-virtual-machine:~$ cp /mnt/hgfs/SharedFolders/led_flash /home/tronlong/docker/dockerfile/
tronlong@tronlong-virtual-machine:~$ cp /mnt/hgfs/SharedFolders/nxp-image-real-time-edge-2.5.0-v1.1-ge46723
d.tar.gz /home/tronlong/docker/dockerfile/
tronlong@tronlong-virtual-machine:~$ ls /home/tronlong/docker/dockerfile/
led_flash  nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz
tronlong@tronlong-virtual-machine:~$
```

图 14

在"/home/tronlong/docker/dockerfile/"目录下，创建 Dockerfile 文件，并编写如下内容。

Host# cd /home/tronlong/docker/dockerfile

Host# gedit Dockerfile

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/docker/dockerfile
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ gedit Dockerfile
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ ls
Dockerfile  led_flash  nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 15

```
Open                    *Dockerfile
                        ~/docker/dockerfile
1 FROM scratch
2
3 # Set work directory
4 WORKDIR /root
5
6 # Decompress the file system
7 ADD nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz /
8
9 # Copy file to image
10 COPY led_flash .
11
12 # Shell command executed when the container is started
13 # CMD ["executable","param1","param2"]
14 CMD ["./led_flash", "-n 1"]
15
```

图 16

Dockerfile 文件内容如下：

FROM scratch #基于空的基础镜像构建新的镜像

Set work directory

WORKDIR /root #为 CMD、COPY 和 AND 设置工作目录

Decompress the file system

ADD nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz / #解压文件系统

Copy file to image

```
COPY led_flash .        #拷贝可执行文件至镜像的"/root/"目录下

# Shell command executed when the container is started
# CMD ["executable","param1","param2"]
CMD ["./led_flash", "-n 1"]        #容器启动时执行的 shell 命令，此处为执行 LED 闪烁程序的命令
```

在 Dockerfile 文件所在的目录下执行如下命令，进行构建镜像。

Host# sudo docker build -t 192.168.13.105:5000/led_flash:v1.0 . //注意命令最后含有"."

Host# sudo docker images //查看已构建的镜像

参数说明：

192.168.13.105：Ubuntu 的 IP 地址；

-t：指定镜像的名字及标签(name:tag)。

```
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker build -t 192.168.13.105:5000/led_flash:v1.0 .
[+] Building 46.5s (7/7) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                          0.0s
 => => transferring dockerfile: 347B                                                                                          0.0s
 => [internal] load .dockerignore                                                                                             0.1s
 => => transferring context: 2B                                                                                               0.0s
 => [1/3] WORKDIR /root                                                                                                       0.0s
 => [internal] load build context                                                                                             4.1s
 => => transferring context: 1.17GB                                                                                           4.1s
 => [2/3] ADD nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz /                                                          37.2s
 => [3/3] COPY led_flash .                                                                                                    0.0s
 => exporting to image                                                                                                        5.0s
 => => exporting layers                                                                                                       5.0s
 => => writing image sha256:088e3632591e6c9dd52c2d2b524350a558df8507f87c64b146b1c80998d0ad51                                  0.0s
 => => naming to 192.168.13.105:5000/led_flash:v1.0                                                                           0.0s
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker images
REPOSITORY                    TAG       IMAGE ID       CREATED          SIZE
192.168.13.105:5000/led_flash v1.0      088e3632591e   43 seconds ago   3.25GB
registry                      2         9363667f8aec   6 months ago     25.4MB
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 17

请执行如下命令，将镜像推送至本地仓库 Registry。

Host# sudo docker push 192.168.13.105:5000/led_flash:v1.0

```
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker push 192.168.13.105:5000/led_flash:v1.0
The push refers to repository [192.168.13.105:5000/led_flash]
Get "https://192.168.13.105:5000/v2/": http: server gave HTTP response to HTTPS client
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 18

Docker 版本从 1.3.X 之后，与 Docker Registry 交互默认使用 https。由于前面搭建的私有仓库仅提供 http 服务，因此当与私有仓库交互时将出现上面的错误提示。可通过修改"/etc/docker/daemon.json"文件来解决该问题，请执行如下命令，192.168.13.105 为 Ubuntu 的 IP。

Host# sudo vi /etc/docker/daemon.json

```
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo vi /etc/docker/daemon.json
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 19

```
1 {
2     "insecure-registries": ["192.168.13.105:5000"]
3 }
4
```

图 20

请在 daemon.json 中添加如下配置内容：

```
{
    "insecure-registries": ["192.168.13.105:5000"]
}
```

执行如下命令，重新启动 Docker、Registry 容器（通过指定 ID）。

Host# sudo /etc/init.d/docker restart

Host# sudo docker ps -a

Host# sudo docker start e2bc50c046d4 //以查询得到的 CONTAINER ID 号为准

```
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo /etc/init.d/docker restart
Restarting docker (via systemctl): docker.service.
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker ps -a
CONTAINER ID   IMAGE        COMMAND                  CREATED          STATUS                      PORTS     NAMES
e2bc50c046d4   registry:2   "/entrypoint.sh /etc…"   14 minutes ago   Exited (2) 10 seconds ago             gracious_leakey
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker start e2bc50c046d4
e2bc50c046d4
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 21

执行如下命令重新推送镜像至本地 Registry。

Host# sudo docker push 192.168.13.105:5000/led_flash:v1.0

```
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker push 192.168.13.105:5000/led_flash:v1.0
The push refers to repository [192.168.13.105:5000/led_flash]
faca5a89f112: Pushed
d3c2732be03e: Pushed
ac1b8914d38c: Pushed
v1.0: digest: sha256:577119663d480ae93ea11c7037b5ee24ca006175430845ccf2377b7b60d26424 size: 945
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

通过浏览器访问"http://192.168.13.105:5000/v2/_catalog"，即可看到当前仓库里已有的镜像。

图 22

3.2 构建多个镜像

请先参考“构建单个镜像”小节完成单个镜像 led_flash 的构建。将产品资料“4-软件资料\Demo\platform-demos\docker\image\”目录下的所有脚本文件拷贝至 Ubuntu 共享目录下，再执行如下命令拷贝至 Docker 工作目录。

Host# cp /mnt/hgfs/SharedFolders/docker_container_1_echo.sh /home/tronlong/docker/dockerfile/

Host# cp /mnt/hgfs/SharedFolders/docker_container_2_echo.sh /home/tronlong/docker/dockerfile/

Host# cp /mnt/hgfs/SharedFolders/docker_container_3_echo.sh /home/tronlong/docker/dockerfile/

```
tronlong@tronlong-virtual-machine:~$ cp /mnt/hgfs/SharedFolders/docker_container_1_echo.sh /home/tronlong/docker/dockerfile/
tronlong@tronlong-virtual-machine:~$ cp /mnt/hgfs/SharedFolders/docker_container_2_echo.sh /home/tronlong/docker/dockerfile/
tronlong@tronlong-virtual-machine:~$ cp /mnt/hgfs/SharedFolders/docker_container_3_echo.sh /home/tronlong/docker/dockerfile/
tronlong@tronlong-virtual-machine:~$ ls /home/tronlong/docker/dockerfile/
docker_container_1_echo.sh  docker_container_2_echo.sh  docker_container_3_echo.sh  Dockerfile  led_flash  nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz
tronlong@tronlong-virtual-machine:~$
```

图 23

在"/home/tronlong/docker/dockerfile/"目录下，修改 Dockerfile 文件。

Host# cd /home/tronlong/docker/dockerfile

Host# gedit Dockerfile

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/docker/dockerfile
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ gedit Dockerfile
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ ls
docker_container_1_echo.sh  docker_container_3_echo.sh  led_flash
docker_container_2_echo.sh  Dockerfile                  nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 24

Open

Dockerfile
~/docker/dockerfile

```
1 FROM scratch
2
3 # Set work directory
4 WORKDIR /root
5
6 # Decompress the file system
7 ADD nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz /
8
9 # Copy file to image
10 COPY docker_container_1_echo.sh .
11
12 # Shell command executed when the container is started
13 # CMD ["executable","param1","param2"]
14 #CMD ["./led_flash", "-n 1"]
15
```

图 25

修改内容如下：

```
COPY docker_container_1_echo.sh .          #拷贝脚本文件至镜像的"/root/"目录下
# CMD ["./led_flash", "-n 1"]              #添加注释符"#"
```

在 Dockerfile 文件所在的目录下执行如下命令，进行构建镜像。

Host# sudo docker build -t 192.168.13.105:5000/docker_container_1_echo.sh:v1.0 ./ /

注意命令最后含有"."

Host# sudo docker images //查看已构建的镜像

参数说明：

192.168.13.105：Ubuntu 的 IP 地址；

-t：指定镜像的名字及标签(name:tag)。

```
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker build -t 192.168.13.105:5000/docker_conta
iner_1_echo.sh:v1.0 .
[sudo] password for tronlong:
[+] Building 0.1s (7/7) FINISHED                                                    docker:default
 => [internal] load build definition from Dockerfile                                          0.0s
 => => transferring dockerfile: 365B                                                          0.0s
 => [internal] load .dockerignore                                                             0.0s
 => => transferring context: 2B                                                               0.0s
 => [1/3] WORKDIR /root                                                                       0.0s
 => [internal] load build context                                                             0.0s
 => => transferring context: 162B                                                             0.0s
 => CACHED [2/3] ADD nxp-image-real-time-edge-2.5.0-v1.1-ge46723d.tar.gz /                    0.0s
 => [3/3] COPY docker_container_1_echo.sh .                                                   0.0s
 => exporting to image                                                                        0.0s
 => => exporting layers                                                                       0.0s
 => => writing image sha256:52606d577ec67bd44685952cf68de95f486262a30137397d2f1e84dedcd0df06  0.0s
 => => naming to 192.168.13.105:5000/docker_container_1_echo.sh:v1.0                          0.0s
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker images
REPOSITORY                                      TAG       IMAGE ID       CREATED          SIZE
192.168.13.105:5000/docker_container_1_echo.sh  v1.0      52606d577ec6   19 seconds ago   3.25GB
192.168.13.105:5000/led_flash                   v1.0      088e3632591e   26 minutes ago   3.25GB
registry                                        2         9363667f8aec   6 months ago     25.4MB
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 26

请执行如下命令，将镜像推送至本地仓库 Registry。

Host# sudo docker push 192.168.13.105:5000/docker_container_1_echo.sh:v1.0

备注：由于构建 Docker 镜像使用的是同一个文件系统，相同的内容将会重叠，因此将会继承第一个 led_flash 镜像已经上传的内容（文件系统）。

```
tronlong@tronlong-virtual-machine:~/docker/dockerfile$ sudo docker push 192.168.13.105:5000/docker_container
_1_echo.sh:v1.0
The push refers to repository [192.168.13.105:5000/docker_container_1_echo.sh]
acf898da5383: Pushed
d3c2732be03e: Mounted from led_flash
ac1b8914d38c: Mounted from led_flash
v1.0: digest: sha256:32b8e0b7e0790656780a069ebd4a4f422bc7bc326f2c8a4aa06b3af7b9d62035 size: 943
tronlong@tronlong-virtual-machine:~/docker/dockerfile$
```

图 27

通过浏览器访问"http://192.168.13.105:5000/v2/_catalog"，即可看到当前仓库里已有的镜像。

图 28

"docker_container_2_echo.sh"、"docker_container_3_echo.sh"文件参考以上类似步骤构建镜像。

重新通过浏览器访问"http://192.168.13.105:5000/v2/_catalog"，即可看到当前仓库里已有多个镜像。

图 29

4 下载镜像并运行容器

4.1 下载单个镜像并运行容器

（1） 请使用网线将评估板 ETH0 网口连接至路由器，并确保和 Ubuntu 处于同一网络下，评估板上电启动，请执行如下命令启动 Docker 服务。

备注：成功启动 Docker 服务需等待几分钟。

Target# systemctl start docker

```
root@IMX8MP-Tronlong:~# systemctl start docker
[ 1530.759380] Bridge firewalling registered
[ 1530.810523] audit: type=1325 audit(1651176450.320:4): table=nat family=2 entries=0 op=xt_register pid=894 comm="ipta
bles"
[ 1530.810572] audit: type=1300 audit(1651176450.320:4): arch=c00000b7 syscall=209 success=yes exit=0 a0=4 a1=0 a2=40 a
3=ffffeb376a30 items=0 ppid=880 pid=894 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(no
ne) ses=4294967295 comm="iptables" exe="/usr/sbin/xtables-legacy-multi" key=(null)
[ 1530.810582] audit: type=1327 audit(1651176450.320:4): proctitle=2F7573722F7362696E2F69707461626C6573002D2D7761697400
2D74006E6174002D4C002D6E
[ 1530.853747] audit: type=1325 audit(1651176450.368:5): table=filter family=2 entries=0 op=xt_register pid=899 comm="i
ptables"
[ 1530.853779] audit: type=1300 audit(1651176450.368:5): arch=c00000b7 syscall=209 success=yes exit=0 a0=4 a1=0 a2=40 a
3=ffffd065a780 items=0 ppid=880 pid=899 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(no
ne) ses=4294967295 comm="iptables" exe="/usr/sbin/xtables-legacy-multi" key=(null)
[ 1530.853798] audit: type=1327 audit(1651176450.368:5): proctitle=2F7573722F7362696E2F69707461626C6573002D2D7761697400
2D4C002D6E
[ 1530.921906] audit: type=1325 audit(1651176450.468:6): table=nat family=2 entries=5 op=xt_replace pid=920 comm="iptab
les"
[ 1530.921924] audit: type=1300 audit(1651176450.468:6): arch=c00000b7 syscall=208 success=yes exit=0 a0=4 a1=0 a2=40 a
3=aaab146e6a60 items=0 ppid=880 pid=920 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(no
ne) ses=4294967295 comm="iptables" exe="/usr/sbin/xtables-legacy-multi" key=(null)
[ 1530.921933] audit: type=1327 audit(1651176450.468:6): proctitle=2F7573722F7362696E2F69707461626C6573002D2D7761697400
2D74006E6174002D4E00444F434B4552
[ 1530.927046] audit: type=1325 audit(1651176450.472:7): table=filter family=2 entries=4 op=xt_replace pid=922 comm="ip
tables"
root@IMX8MP-Tronlong:~#
```

图 30

执行如下命令查看是否启动成功。

Target# ifconfig

```
root@IMX8MP-Tronlong:~# ifconfig
docker0: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet 172.17.0.1  netmask 255.255.0.0  broadcast 172.17.255.255
        ether 02:42:bf:ba:8f:27  txqueuelen 0  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
        inet 192.168.13.28  netmask 255.255.255.0  broadcast 192.168.13.255
        inet6 fe80::6418:3dff:fe84:d6b7  prefixlen 64  scopeid 0x20<link>
        ether 66:18:3d:84:d6:b7  txqueuelen 1000  (Ethernet)
        RX packets 1269  bytes 251196 (245.3 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 157  bytes 24515 (23.9 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

eth1: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        ether 76:30:92:22:c7:be  txqueuelen 1000  (Ethernet)
        RX packets 0  bytes 0 (0.0 B)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 0  bytes 0 (0.0 B)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0
        device interrupt 61

eth2: flags=4099<UP,BROADCAST,MULTICAST>  mtu 1500
        inet6 fe80::2e0:99ff:fea2:29ce  prefixlen 64  scopeid 0x20<link>
        ether 00:e0:99:a2:29:ce  txqueuelen 1000  (Ethernet)
        RX packets 9506  bytes 1867328 (1.7 MiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 215  bytes 30368 (29.6 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
        inet 127.0.0.1  netmask 255.0.0.0
        inet6 ::1  prefixlen 128  scopeid 0x10<host>
        loop  txqueuelen 1000  (Local Loopback)
        RX packets 102  bytes 9343 (9.1 KiB)
        RX errors 0  dropped 0  overruns 0  frame 0
        TX packets 102  bytes 9343 (9.1 KiB)
        TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

root@IMX8MP-Tronlong:~#
```

图 31

执行如下命令下载镜像并列举出下载后的镜像。

Target# docker pull 192.168.13.105:5000/led_flash:v1.0 //下载镜像

Target# docker images //列举已下载的镜像

```
root@IMX8MP-Tronlong:~# docker pull 192.168.13.105:5000/led_flash:v1.0
v1.0: Pulling from led_flash
957e33970d4b: Pull complete
4041a277a2a2: Extracting [==========================================>          ]  969.3MB/1.191GB
f3bcc25e71d0: Download complete
[ 2882.404464] kauditd_printk_skb: 107 callbacks suppressed
[ 2882.404475] audit: type=1006 audit(1651177801.948:70): pid=1213 uid=0 old-auid=4294967295 auid=0 tty=(none) old-ses=
4294967295 ses=3 res=1
[ 2882.404488] audit: type=1300 audit(1651177801.948:70): arch=c00000b7 syscall=64 success=yes exit=1 a0=3 a1=ffffe8c5a
e80 a2=1 a3=ffffad026020 items=0 ppid=246 pid=1213 auid=0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(
4041a277a2a2: Pull complete
f3bcc25e71d0: Pull complete
Digest: sha256:577119663d480ae93ea11c7037b5ee24ca006175430845ccf2377b7b60d26424
Status: Downloaded newer image for 192.168.13.105:5000/led_flash:v1.0
192.168.13.105:5000/led_flash:v1.0
root@IMX8MP-Tronlong:~# docker images
REPOSITORY                      TAG       IMAGE ID       CREATED                 SIZE
192.168.13.105:5000/led_flash   v1.0      088e3632591e   Less than a second ago  3.25GB
root@IMX8MP-Tronlong:~#
```

图 32

若出现如下错误，请执行如下命令，通过修改"/etc/docker/daemon.json"文件来解决此问题，修改保存后，重启 Docker 服务。

```
root@IMX8MP-Tronlong:~# docker pull 192.168.13.105:5000/led_flash:v1.0
Error response from daemon: Get "https://192.168.13.105:5000/v2/": http: server gave HTTP response to HTTPS client
root@IMX8MP-Tronlong:~#
```

图 33

Target# vi /etc/docker/daemon.json

```
root@IMX8MP-Tronlong:~# vi /etc/docker/daemon.json
root@IMX8MP-Tronlong:~#
```

图 34

请在 daemon.json 中添加如下配置内容：

```
{
    "insecure-registries": ["192.168.13.105:5000"]
}
```

```
{
    "insecure-registries": ["192.168.13.105:5000"]
}
~
```

图 35

Target# systemctl restart docker.service

```
root@IMX8MP-Tronlong:~# systemctl restart docker.service
[ 2527.341058] kauditd_printk_skb: 71 callbacks suppressed
[ 2527.341069] audit: type=1325 audit(1651177446.884:31): table=nat family=2 entries=11 op=xt_replace pid=1062 comm="ip
tables"
[ 2527.341082] audit: type=1300 audit(1651177446.884:31): arch=c00000b7 syscall=208 success=yes exit=0 a0=4 a1=0 a2=40
a3=aaaad71e99c0 items=0 ppid=1047 pid=1062 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=
(none) ses=4294967295 comm="iptables" exe="/usr/sbin/xtables-legacy-multi" key=(null)
[ 2527.341091] audit: type=1327 audit(1651177446.884:31): proctitle=2F7573722F7362696E2F69707461626C6573002D2D776169740
02D74006E6174002D4400505245524F5554494E47002D6D006164647274797065002D2D6473742D74797065004C4F43414C002D6A00444F434B4552
[ 2527.345363] audit: type=1325 audit(1651177446.888:32): table=nat family=2 entries=10 op=xt_replace pid=1063 comm="ip
tables"
[ 2527.345384] audit: type=1300 audit(1651177446.888:32): arch=c00000b7 syscall=208 success=yes exit=0 a0=4 a1=0 a2=40
a3=aaaafc9267f0 items=0 ppid=1047 pid=1063 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=
(none) ses=4294967295 comm="iptables" exe="/usr/sbin/xtables-legacy-multi" key=(null)
[ 2527.345393] audit: type=1327 audit(1651177446.888:32): proctitle=2F7573722F7362696E2F69707461626C6573002D2D776169740
02D74006E6174002D44004F5554505554002D6D006164647274797065002D2D6473742D74797065004C4F43414C0000002D2D647374003132372E30
2E302E302F38002D6A00444F434B4552
[ 2527.358286] audit: type=1325 audit(1651177446.904:33): table=nat family=2 entries=9 op=xt_replace pid=1067 comm="ipt
ables"
[ 2527.358304] audit: type=1300 audit(1651177446.904:33): arch=c00000b7 syscall=208 success=yes exit=0 a0=4 a1=0 a2=40
a3=aaab06739ec0 items=0 ppid=1047 pid=1067 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=
(none) ses=4294967295 comm="iptables" exe="/usr/sbin/xtables-legacy-multi" key=(null)
[ 2527.358313] audit: type=1327 audit(1651177446.904:33): proctitle=2F7573722F7362696E2F69707461626C6573002D2D776169740
02D74006E6174002D4600444F434B4552
[ 2527.361991] audit: type=1325 audit(1651177446.908:34): table=nat family=2 entries=8 op=xt_replace pid=1068 comm="ipt
ables"
root@IMX8MP-Tronlong:~#
```

图 36

（2） 创建并启动容器。

Target# docker run -d --name led_flash -v /sys/:/sys 192.168.13.105:5000/led_flash:v1.0

参数说明：

-d：以交互模式运行容器，允许用户与容器进行交互，后台运行容器；

--name：为容器指定一个名称；

-v：映射卷（将主机的目录"/sys"映射至容器的"/sys"）；

192.168.12.105:5000/led_flash:v1.0：镜像名称和标签。

```
root@IMX8MP-Tronlong:~# docker run -d --name led_flash -v /sys/:/sys 192.168.13.105:5000/led_flash:v1.0
WARNING: The requested image's platform (linux/amd64) does not match the detected host platform (linux/arm64/v8) and no
 specific platform was requested
f6cd549fb9a75930ef2ea20a8813c7e8dea154b5053906ec1e85889becd8ca2c
[ 3420.383504] docker0: port 1(veth5064ca8) entered blocking state
[ 3420.383532] docker0: port 1(veth5064ca8) entered disabled state
[ 3420.384461] device veth5064ca8 entered promiscuous mode
[ 3420.384530] audit: type=1700 audit(1651178339.928:71): dev=veth5064ca8 prom=256 old_prom=0 auid=4294967295 uid=0 gid
=0 ses=4294967295
[ 3420.399326] audit: type=1300 audit(1651178339.928:71): arch=c00000b7 syscall=206 success=yes exit=40 a0=e a1=4000f14
f90 a2=28 a3=0 items=0 ppid=1 pid=1047 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(non
e) ses=4294967295 comm="dockerd" exe="/usr/bin/dockerd" key=(null)
[ 3420.399821] audit: type=1327 audit(1651178339.928:71): proctitle=2F7573722F62696E2F646F636B657264002D480066643A2F2F
[ 3421.151946] eth0: renamed from vethc9836c6
[ 3421.172309] IPv6: ADDRCONF(NETDEV_CHANGE): veth5064ca8: link becomes ready
[ 3421.172412] docker0: port 1(veth5064ca8) entered blocking state
[ 3421.172420] docker0: port 1(veth5064ca8) entered forwarding state
[ 3421.172609] IPv6: ADDRCONF(NETDEV_CHANGE): docker0: link becomes ready
root@IMX8MP-Tronlong:~#
```

图 37

容器启动完成后，评估底板的 1 个用户可编程指示灯闪烁。执行如下命令，可查看当前容器占用资源情况。"MEM USAGE"表示容器占用内存；"LIMIT"表示容器可使用的最大内存。

Target# docker stats

```
root@IMX8MP-Tronlong:~# docker stats
```

图 38

```
CONTAINER ID   NAME        CPU %     MEM USAGE / LIMIT     MEM %     NET I/O       BLOCK I/O   PIDS
f6cd549fb9a7   led_flash   0.59%     3.043MiB / 1.653GiB   0.18%     9.11kB / 0B   0B / 0B     1
```

图 39

（3） 容器的基本操作

执行如下命令，查看正在运行的容器并进入容器。

Target# docker ps

Target# docker exec -it led_flash /bin/bash

参数说明：

-i：以交互模式运行容器，通常与-t 同时使用；

-t：为容器重新分配一个伪输入终端，通常与-i 同时使用；

led_flash：容器名称，以实际"NAME"为准；

/bin/bash：在容器内执行"/bin/bash"命令。

```
root@IMX8MP-Tronlong:~# docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS         PORTS     NAMES
f6cd549fb9a7   192.168.13.105:5000/led_flash:v1.0   "./led_flash '-n 1'"     4 minutes ago  Up 4 minutes             led_flash
root@IMX8MP-Tronlong:~# docker exec -it led_flash /bin/bash
bash-5.1#
```

图 40

执行如下命令，退出容器。

Bash# exit

```
root@IMX8MP-Tronlong:~# docker exec -it led_flash /bin/bash
bash-5.1#
bash-5.1# exit
exit
root@IMX8MP-Tronlong:~#
```

图 41

执行如下命令，停止容器。此时，评估底板的 1 个用户可编程指示灯不再闪烁。

Target# docker stop led_flash

```
root@IMX8MP-Tronlong:~# docker stop led_flash
[ 3867.999444] docker0: port 1(veth5064ca8) entered disabled state
[ 3867.999797] vethc9836c6: renamed from eth0
[ 3868.062037] docker0: port 1(veth5064ca8) entered disabled state
[ 3868.063991] device veth5064ca8 left promiscuous mode
[ 3868.064025] docker0: port 1(veth5064ca8) entered disabled state
[ 3868.064128] audit: type=1700 audit(1651178787.604:72): dev=veth5064ca8 prom=0 old_prom=256 auid=4294967295 uid=0 gid=0 ses=
4294967295
[ 3868.117797] audit: type=1300 audit(1651178787.604:72): arch=c00000b7 syscall=206 success=yes exit=32 a0=e a1=40003b5780 a2=
20 a3=0 items=0 ppid=1 pid=1047 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967
295 comm="dockerd" exe="/usr/bin/dockerd" key=(null)
[ 3868.118247] audit: type=1327 audit(1651178787.604:72): proctitle=2F7573722F62696E2F646F636B657264002D480066643A2F2F
led_flash
root@IMX8MP-Tronlong:~#
```

图 42

执行如下命令，启动容器。此时，评估底板的 1 个用户可编程指示灯闪烁。

Target# docker start led_flash

```
root@IMX8MP-Tronlong:~# docker start led_flash
[ 4030.688378] docker0: port 1(veth1a13b8f3) entered blocking state
[ 4030.688397] docker0: port 1(veth1a13b8f3) entered disabled state
[ 4030.688958] device veth1a13b8f3 entered promiscuous mode
[ 4030.689025] audit: type=1700 audit(1651178950.232:73): dev=veth1a13b8f3 prom=256 old_prom=0 auid=4294967295 uid=0 gid=0 ses=
4294967295
[ 4030.689188] audit: type=1300 audit(1651178950.232:73): arch=c00000b7 syscall=206 success=yes exit=40 a0=e a1=4000f14ed0 a2=
28 a3=0 items=0 ppid=1 pid=1047 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967
295 comm="dockerd" exe="/usr/bin/dockerd" key=(null)
[ 4030.689198] audit: type=1327 audit(1651178950.232:73): proctitle=2F7573722F62696E2F646F636B657264002D480066643A2F2F
[ 4030.689364] docker0: port 1(veth1a13b8f3) entered blocking state
[ 4030.689371] docker0: port 1(veth1a13b8f3) entered forwarding state
[ 4030.721883] docker0: port 1(veth1a13b8f3) entered disabled state
[ 4031.231518] eth0: renamed from veth19ff241
[ 4031.250355] IPv6: ADDRCONF(NETDEV_CHANGE): veth1a13b8f3: link becomes ready
[ 4031.250452] docker0: port 1(veth1a13b8f3) entered blocking state
[ 4031.250460] docker0: port 1(veth1a13b8f3) entered forwarding state
led_flash
root@IMX8MP-Tronlong:~#
```

图 43

4.2 下载多个镜像并运行容器

（1） 请使用网线将评估板 ETH0 网口连接至路由器，并确保和 Ubuntu 处于同一网络下，评估板上电启动，执行如下命令下载镜像并列举出下载后的镜像。

Target# docker pull 192.168.13.105:5000/docker_container_1_echo.sh:v1.0 // 下载镜像

Target# docker pull 192.168.13.105:5000/docker_container_2_echo.sh:v1.0

Target# docker pull 192.168.13.105:5000/docker_container_3_echo.sh:v1.0

Target# docker images //列举已下载的镜像

备注：由于 led_flash 镜像已在“下载单个镜像并运行容器”小节中下载、创建并启动容器，因此本小节不再演示。

```
root@IMX8MP-Tronlong:~# docker pull 192.168.13.105:5000/docker_container_1_echo.sh:v1.0
v1.0: Pulling from docker_container_1_echo.sh
957e33970d4b: Already exists
4041a277a2a2: Already exists
9b241581c23f: Pull complete
Digest: sha256:32b8e0b7e0790656780a069ebd4a4f422bc7bc326f2c8a4aa06b3af7b9d62035
Status: Downloaded newer image for 192.168.13.105:5000/docker_container_1_echo.sh:v1.0
192.168.13.105:5000/docker_container_1_echo.sh:v1.0
root@IMX8MP-Tronlong:~# docker pull 192.168.13.105:5000/docker_container_2_echo.sh:v1.0
v1.0: Pulling from docker_container_2_echo.sh
957e33970d4b: Already exists
4041a277a2a2: Already exists
7dbbb0a0c11f: Pull complete
Digest: sha256:dea11bba9f17bfb813f1d94e8671f916fb460d0938e981ca8bd43d0bb9d1d00e
Status: Downloaded newer image for 192.168.13.105:5000/docker_container_2_echo.sh:v1.0
192.168.13.105:5000/docker_container_2_echo.sh:v1.0
root@IMX8MP-Tronlong:~# docker pull 192.168.13.105:5000/docker_container_3_echo.sh:v1.0
v1.0: Pulling from docker_container_3_echo.sh
957e33970d4b: Already exists
4041a277a2a2: Already exists
f7775ef86b5e: Pull complete
Digest: sha256:bb5eaefe9aae29f3b5426911ca24546ef12f91726708887a732d600bf0ae6e87
Status: Downloaded newer image for 192.168.13.105:5000/docker_container_3_echo.sh:v1.0
192.168.13.105:5000/docker_container_3_echo.sh:v1.0
root@IMX8MP-Tronlong:~# docker images
REPOSITORY                                      TAG       IMAGE ID       CREATED                  SIZE
192.168.13.105:5000/docker_container_3_echo.sh  v1.0      0552f9611db2   Less than a second ago   3.25GB
192.168.13.105:5000/docker_container_2_echo.sh  v1.0      6f84bf6d7998   Less than a second ago   3.25GB
192.168.13.105:5000/docker_container_1_echo.sh  v1.0      52606d577ec6   Less than a second ago   3.25GB
192.168.13.105:5000/led_flash                   v1.0      088e3632591e   Less than a second ago   3.25GB
root@IMX8MP-Tronlong:~#
```

图 44

（2） 执行如下命令，创建并启动容器。

Target# docker run -itd --platform linux/amd64 --name docker_container_1_echo 192.168.13.105:5000/docker_container_1_echo.sh:v1.0 /bin/bash

Target# docker run -itd --platform linux/amd64 --name docker_container_2_echo 192.168.13.105:5000/docker_container_2_echo.sh:v1.0 /bin/bash

Target# docker run -itd --platform linux/amd64 --name docker_container_3_echo 192.168.13.105:5000/docker_container_3_echo.sh:v1.0 /bin/bash

参数说明：

-itd：以交互模式运行容器，允许用户与容器进行交互，后台运行容器；

--platform：指定容器运行的平台；

--name：为容器指定一个名称；

192.168.12.105:5000/docker_container_x_echo.sh:v1.0：镜像名称和标签。

/bin/bash：执行/bin/bash 命令阻塞 docker 容器，防止容器自动退出，以便能进入容器运行脚本。

```
root@IMX8MP-Tronlong:~# docker run -itd --platform linux/amd64 --name docker_container_1_echo 192.168.13.105:5000/docker_conta
iner_1_echo.sh:v1.0 /bin/bash
d43c8a7f86843088656d1273d279b344a6ff8d5f891d9eaf6e2220fa07758991
[ 4773.508408] docker0: port 1(veth1d57755) entered blocking state
[ 4773.508424] docker0: port 1(veth1d57755) entered disabled state
[ 4773.508700] device veth1d57755 entered promiscuous mode
[ 4773.508786] audit: type=1700 audit(1651179693.052:75): dev=veth1d57755 prom=256 old_prom=0 auid=4294967295 uid=0 gid=0 ses=
4294967295
[ 4773.509079] audit: type=1300 audit(1651179693.052:75): arch=c00000b7 syscall=206 success=yes exit=40 a0=e a1=400055aa80 a2=
28 a3=0 items=0 ppid=1 pid=1047 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967
295 comm="dockerd" exe="/usr/bin/dockerd" key=(null)
[ 4773.509120] audit: type=1327 audit(1651179693.052:75): proctitle=2F7573722F62696E2F646F636B657264002D480066643A2F2F
[ 4773.509387] docker0: port 1(veth1d57755) entered blocking state
[ 4773.509395] docker0: port 1(veth1d57755) entered forwarding state
[ 4773.523654] docker0: port 1(veth1d57755) entered disabled state
[ 4774.058643] eth0: renamed from veth1f01cd5
[ 4774.078310] IPv6: ADDRCONF(NETDEV_CHANGE): veth1d57755: link becomes ready
[ 4774.078413] docker0: port 1(veth1d57755) entered blocking state
[ 4774.078422] docker0: port 1(veth1d57755) entered forwarding state
root@IMX8MP-Tronlong:~# docker run -itd --platform linux/amd64 --name docker_container_2_echo 192.168.13.105:5000/docker_conta
iner_2_echo.sh:v1.0 /bin/bash
34c268163e7fef56212d3e380bff142f82487329419598925b5d20a3053f6fb5
[ 4795.775057] docker0: port 2(vethb7a5cb8) entered blocking state
[ 4795.775072] docker0: port 2(vethb7a5cb8) entered disabled state
[ 4795.775573] device vethb7a5cb8 entered promiscuous mode
[ 4795.775655] audit: type=1700 audit(1651179715.320:76): dev=vethb7a5cb8 prom=256 old_prom=0 auid=4294967295 uid=0 gid=0 ses=
4294967295
[ 4795.776256] audit: type=1300 audit(1651179715.320:76): arch=c00000b7 syscall=206 success=yes exit=40 a0=e a1=4000483770 a2=
28 a3=0 items=0 ppid=1 pid=1047 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967
295 comm="dockerd" exe="/usr/bin/dockerd" key=(null)
[ 4795.776276] audit: type=1327 audit(1651179715.320:76): proctitle=2F7573722F62696E2F646F636B657264002D480066643A2F2F
[ 4796.323347] eth0: renamed from vethbc2c7d7
[ 4796.346330] IPv6: ADDRCONF(NETDEV_CHANGE): vethb7a5cb8: link becomes ready
[ 4796.346428] docker0: port 2(vethb7a5cb8) entered blocking state
[ 4796.346438] docker0: port 2(vethb7a5cb8) entered forwarding state
root@IMX8MP-Tronlong:~# docker run -itd --platform linux/amd64 --name docker_container_3_echo 192.168.13.105:5000/docker_conta
iner_3_echo.sh:v1.0 /bin/bash
3b1807de87dfe85714c62a58f1c1645916d8ba57db62bb413e565d2fb4a1f5b1
[ 4802.155159] docker0: port 3(veth17ad0b5) entered blocking state
[ 4802.155175] docker0: port 3(veth17ad0b5) entered disabled state
[ 4802.155642] device veth17ad0b5 entered promiscuous mode
[ 4802.155782] audit: type=1700 audit(1651179721.700:77): dev=veth17ad0b5 prom=256 old_prom=0 auid=4294967295 uid=0 gid=0 ses=
4294967295
[ 4802.156640] audit: type=1300 audit(1651179721.700:77): arch=c00000b7 syscall=206 success=yes exit=40 a0=e a1=4000a725d0 a2=
28 a3=0 items=0 ppid=1 pid=1047 auid=4294967295 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=(none) ses=4294967
295 comm="dockerd" exe="/usr/bin/dockerd" key=(null)
[ 4802.157012] audit: type=1327 audit(1651179721.700:77): proctitle=2F7573722F62696E2F646F636B657264002D480066643A2F2F
[ 4802.706444] eth0: renamed from vethca86cfb
[ 4802.726328] IPv6: ADDRCONF(NETDEV_CHANGE): veth17ad0b5: link becomes ready
[ 4802.726438] docker0: port 3(veth17ad0b5) entered blocking state
[ 4802.726447] docker0: port 3(veth17ad0b5) entered forwarding state
root@IMX8MP-Tronlong:~#
```

图 45

执行如下命令，查看正在运行的容器。

Target# docker ps

```
root@IMX8MP-Tronlong:~# docker ps
CONTAINER ID   IMAGE                                                COMMAND                  CREATED          STATUS          PORTS     NAMES
3b1807de87df   192.168.13.105:5000/docker_container_3_echo.sh:v1.0   "/bin/bash"              3 minutes ago    Up 3 minutes              docker_container_3_echo
34c268163e7f   192.168.13.105:5000/docker_container_2_echo.sh:v1.0   "/bin/bash"              3 minutes ago    Up 3 minutes              docker_container_2_echo
d43c8a7f8684   192.168.13.105:5000/docker_container_1_echo.sh:v1.0   "/bin/bash"              3 minutes ago    Up 3 minutes              docker_container_1_echo
f6cd549fb9a7   192.168.13.105:5000/led_flash:v1.0                   "./led_flash '-n 1'"     26 minutes ago   Up 43 seconds             led_flash
root@IMX8MP-Tronlong:~#
```

图 46

容器启动完成后，可执行如下命令查看当前容器占用资源情况。

Target# docker stats

```
root@IMX8MP-Tronlong:~# docker stats
```

图 47

```
CONTAINER ID   NAME                    CPU %     MEM USAGE / LIMIT     MEM %     NET I/O       BLOCK I/O   PIDS
3b1807de87df   docker_container_3_echo   0.00%     720KiB / 1.653GiB     0.04%     5.65kB / 0B   0B / 0B     1
34c268163e7f   docker_container_2_echo   0.00%     712KiB / 1.653GiB     0.04%     5.72kB / 0B   0B / 0B     1
d43c8a7f8684   docker_container_1_echo   0.00%     1.457MiB / 1.653GiB   0.09%     6.9kB / 0B    0B / 0B     1
f6cd549fb9a7   led_flash               0.58%     656KiB / 1.653GiB     0.04%     4.26kB / 0B   0B / 0B     1
```

图 48

（3） 容器的基本操作

执行如下命令，查看正在运行的容器，并进入"name docker_container_1_echo"容器。

备注："name docker_container_2_echo"、"name docker_container_3_echo"容器可参考该操作进行测试。

Target# docker ps -a

Target# docker exec -it docker_container_1_echo /bin/bash

参数说明：

-i：以交互模式运行容器，通常与-t 同时使用；

-t：为容器重新分配一个伪输入终端，通常与-i 同时使用；

docker_container_1_echo：容器名称；

/bin/bash：在容器内执行"/bin/bash"命令。

```
root@IMX8MP-Tronlong:~# docker ps -a
CONTAINER ID   IMAGE                                                COMMAND                  CREATED          STATUS          PORTS     NAMES
3b1807de87df   192.168.13.105:5000/docker_container_3_echo.sh:v1.0   "/bin/bash"              5 minutes ago    Up 5 minutes              docker_container_3_echo
34c2681636f7   192.168.13.105:5000/docker_container_2_echo.sh:v1.0   "/bin/bash"              5 minutes ago    Up 5 minutes              docker_container_2_echo
d43c8a7f8684   192.168.13.105:5000/docker_container_1_echo.sh:v1.0   "/bin/bash"              6 minutes ago    Up 6 minutes              docker_container_1_echo
f6cd549fb9a7   192.168.13.105:5000/led_flash:v1.0                   "./led_flash '-n 1'"     28 minutes ago   Up 3 minutes              led_flash
root@IMX8MP-Tronlong:~# docker exec -it docker_container_1_echo /bin/bash
bash-5.1#
```

图 49

执行如下命令，可查看脚本打印信息。

Target# ./docker_container_1_echo.sh

```
root@IMX8MP-Tronlong:~# docker exec -it docker_container_1_echo /bin/bash
bash-5.1#
bash-5.1# ls
docker_container_1_echo.sh
bash-5.1# ./docker_container_1_echo.sh
This is docker container 1.
bash-5.1#
```

图 50

执行如下命令，退出该容器。

Bash# exit

```
root@IMX8MP-Tronlong:~# docker exec -it docker_container_1_echo /bin/bash
bash-5.1#
bash-5.1# ls
docker_container_1_echo.sh
bash-5.1# ./docker_container_1_echo.sh
This is docker container 1.
bash-5.1# exit
exit
root@IMX8MP-Tronlong:~#
```

图 51

5 查看磁盘占用资源情况

5.1 未下载 Docker 镜像

进入评估板文件系统，执行如下命令，查看当前系统未下载 Docker 镜像时的磁盘使用情况。

Target# df -lh

```
root@IMX8MP-Tronlong:~# df -lh
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        12G  6.5G  4.6G  59% /
devtmpfs        365M  4.0K  365M   1% /dev
tmpfs           847M     0  847M   0% /dev/shm
tmpfs           339M  9.3M  330M   3% /run
tmpfs           4.0M     0  4.0M   0% /sys/fs/cgroup
tmpfs           847M   92K  847M   1% /tmp
tmpfs           847M  300K  846M   1% /var/volatile
/dev/mmcblk2p3   89M   14K   82M   1% /run/media/mmcblk2p3
/dev/mmcblk2p2  3.9G  3.3G  419M  89% /run/media/mmcblk2p2
/dev/mmcblk2p1  127M   63M   64M  50% /run/media/mmcblk2p1
tmpfs           170M  4.0K  170M   1% /run/user/0
/dev/mmcblk1p3   12G  3.3G  7.8G  30% /run/media/mmcblk1p3
/dev/mmcblk1p1  6.0G   63M  5.9G   2% /run/media/mmcblk1p1
root@IMX8MP-Tronlong:~#
```

图 52

5.2 已下载 Docker 镜像

执行如下命令，查看系统已下载 4 个 Docker 镜像时的磁盘使用情况以及 Docker 磁盘空间使用情况。

Target# df -lh

Target# docker system df //查看 Docker 磁盘空间使用情况

```
root@IMX8MP-Tronlong:~# df -lh
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        12G  6.5G  4.6G  59% /
devtmpfs        365M  4.0K  365M   1% /dev
tmpfs           847M     0  847M   0% /dev/shm
tmpfs           339M  9.5M  330M   3% /run
tmpfs           4.0M     0  4.0M   0% /sys/fs/cgroup
tmpfs           847M   92K  847M   1% /tmp
tmpfs           847M  336K  846M   1% /var/volatile
/dev/mmcblk2p3   89M   14K   82M   1% /run/media/mmcblk2p3
/dev/mmcblk2p2  3.9G  3.3G  419M  89% /run/media/mmcblk2p2
/dev/mmcblk2p1  127M   63M   64M  50% /run/media/mmcblk2p1
tmpfs           170M  4.0K  170M   1% /run/user/0
/dev/mmcblk1p3   12G  3.3G  7.8G  30% /run/media/mmcblk1p3
/dev/mmcblk1p1  6.0G   63M  5.9G   2% /run/media/mmcblk1p1
overlay          12G  6.5G  4.6G  59% /var/lib/docker/overlay2/20d4bab8556554ceaed10fb642244149c5dde844032001e8ea485df99f398235/merged
overlay          12G  6.5G  4.6G  59% /var/lib/docker/overlay2/90f1e57cf9c04570fb88578a3a4bc044f233bb0edb3e4ac9407489c44c25f186/merged
overlay          12G  6.5G  4.6G  59% /var/lib/docker/overlay2/a8eef1c79af78a010c4198284bfdda0c537be6f4484c32654f5c56bf34c906d6/merged
overlay          12G  6.5G  4.6G  59% /var/lib/docker/overlay2/4bbe99181aeb66b1e1203794b5696bc55f3bf99cfe058f1668b20a45cc863334/merged
root@IMX8MP-Tronlong:~#
```

图 53

```
root@IMX8MP-Tronlong:~# docker system df
TYPE            TOTAL     ACTIVE    SIZE      RECLAIMABLE
Images          4         4         3.251GB   3.251GB (99%)
Containers      4         4         42B       0B (0%)
Local Volumes   0         0         0B        0B
Build Cache     0         0         0B        0B
root@IMX8MP-Tronlong:~#
```

图 54

可以查看到 4 个 Docker 镜像与容器占用内存的总大小约为 3.251GByte。由于构建 Docker 镜像的文件系统是同一个，而 Docker 镜像中相同的内容会重叠在一起，不需重新占用内存，因此 4 个 Docker 镜像和容器占用的内存总大小约为 3.251GByte。

更多帮助

销售邮箱：sales@tronlong.com

技术邮箱：support@tronlong.com

创龙总机：020-8998-6280

技术热线：020-3893-9734

创龙官网：www.tronlong.com

技术论坛：www.51ele.net

官方商城：tronlong.tmall.com